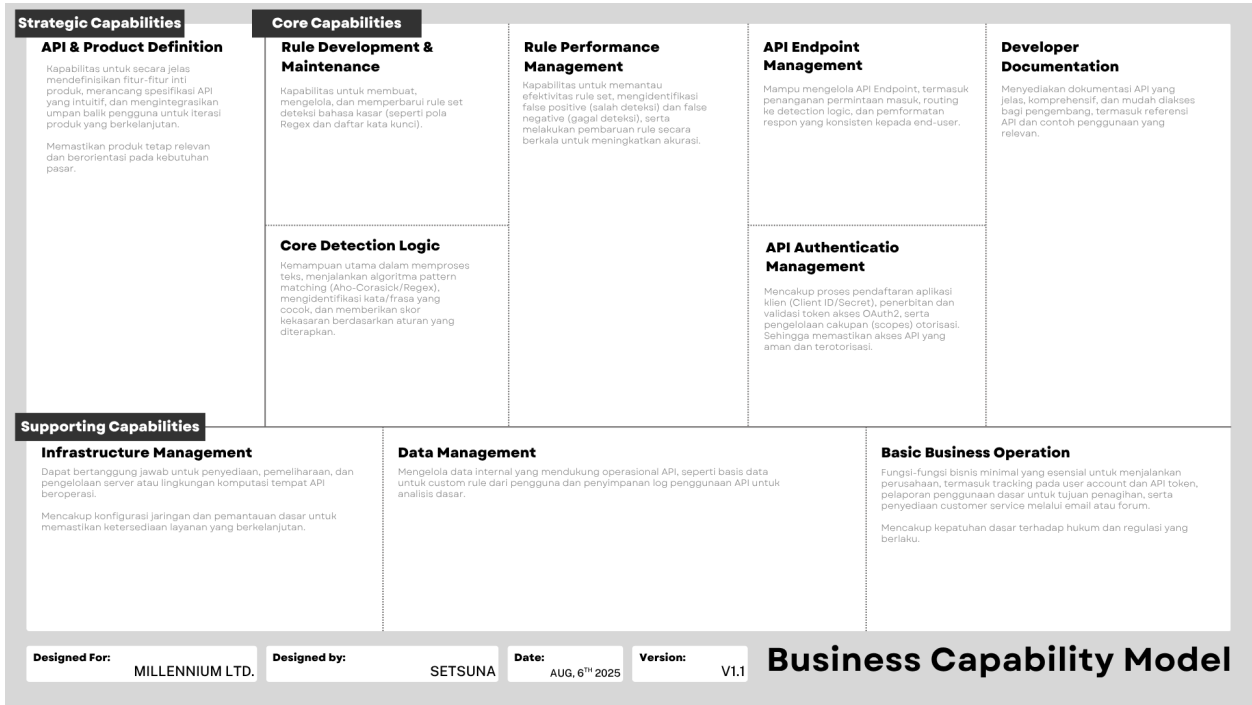


Business Capability Model



Gambar 1.1. Diagram *Business Capability Model* (Sumber: Dokumentasi Penulis)

Analisis Teknologi yang Dibutuhkan

Pengembangan dan operasional layanan (SaaS) Veritas Shield oleh Millenium Ltd. akan memerlukan kombinasi teknologi di berbagai lapisan. Berikut adalah analisis teknologi utama yang dibutuhkan:

1. Teknologi *Pattern Matching & Rule Management*

- a. **Programming Language:** Python dipilih karena kemudahan penggunaan, ekosistem yang kaya, dan dukungan bawaan untuk *regex*.
- b. **Regex Library:** Re (modul bawaan Python).
- c. **Aho-Corasick Library:** Dapat menggunakan *library* Python yang sudah ada (pyahocorasick, flashtext).
- d. **Struktur Data untuk Rule:**
 - i. Daftar kata kasar inti dan user customized disimpan di dalam satu database.
- e. **Rule Management System:** Manual atau melalui API untuk *user customized list*.

2. Teknologi Backend & API

- a. **Programming Language:** Python.
- b. **Web Framework:**
 - i. **FastAPI:** Merupakan pilihan utama karena ringan, cepat, modern, dan secara otomatis menghasilkan dokumentasi API (OpenAPI/Swagger UI) yang akan sangat menghemat waktu saat membuat dokumentasi.
 - ii. **Flask:** Alternatif yang lebih minimalis.
- c. **Authentication & Authorization (OAuth2):**
 - i. **OAuth2 Library:** Menggunakan *library* Python untuk implementasi OAuth2 (Authlib atau Flask-OAuthlib jika menggunakan Flask) untuk membantu dan mengelola alur validasi.
 - ii. **JSON Web Tokens (JWT):** Token akses akan diimplementasikan sebagai JWT untuk validasi yang efisien tanpa perlu *lookup* database pada setiap permintaan API menggunakan *library* PyJWT.
- d. **Rate Limiting:** Dapat diimplementasikan secara sederhana di dalam kode aplikasi menggunakan *in-memory cache* (*library* cachetools, functools.lru_cache di Python) atau dengan Redis sebagai alternatif dengan kompleksitas minimal.

3. Teknologi Database

- a. **Database Utama:**
 - i. **PostgreSQL:** *Relational* database yang kuat untuk menyimpan API keys, daftar kata *user customized*, dan *usage log*.
 - ii. **SQLite:** Alternatif yang dapat digunakan apabila ingin database yang *serverless*, mudah dipahami.
- b. **ORM/Database Library:** SQLAlchemy (Python *library*) atau *library* bawaan *framework* (ORM Flask/FastAPI) untuk berinteraksi dengan database.

4. Cloud Infrastructure (Hosting)

- a. **Penyedia Cloud:**
 - i. **Render (Free Tier):** Menawarkan *free tier* untuk layanan web dan database.
 - ii. **Railway (Free Tier):** Pilihan lain dengan *free tier* yang murah.
 - iii. **DigitalOcean Droplet:** Menawarkan kontrol lebih besar dengan biaya minimal (sekitar \$4-6/bulan).
 - iv. **Local Deployment (Ngrok):** Untuk demo atau pengujian awal, bisa menjalankan API secara lokal dan mengesposnya ke internet menggunakan Ngrok, meskipun ini bukan solusi untuk produksi.
- b. **Layanan Komputasi:** Satu instance VM kecil atau *container* tunggal.
- c. **CI/CD:** Manual deployment atau GitHub Actions dasar untuk otomatisasi *build* dan *deploy* yang sangat sederhana.

5. Teknologi DevOps & Pemantauan

- a. **Deployment:** Manual git push ke *repository* dan langsung di handle Render/Railway, atau SSH ke VM untuk git pull.
- b. **Pemantauan & Logging:** Log aplikasi ke *console/file*, dan pantau melalui *dashboard cloud service*.

6. Teknologi Developer Portal

- a. **Dokumentasi API Otomatis:** FastAPI akan menghasilkan endpoint `/docs` (Swagger UI) secara otomatis. Merupakan cara termudah dan tercepat untuk menyediakan dokumentasi.
- b. **Static Webpage:** Halaman HTML/Markdown sederhana di GitHub Pages atau Netlify untuk informasi dasar dan pendaftaran aplikasi klien.